Simulation-informed transfer learning improves low-data nanobody thermal stability prediction

Taihei Murakami†[2], Kentaro Sasaki†[2], Soichiro Oda[2], Kazuma Okada[2], and Yasuhiro Matsunaga*[1,2]

[1]RIKEN Center for Computational Science, Kobe, Japan
[2]Saitama University, Saitama, Japan

†These authors contributed equally to this work.

Nanobody engineering depends on thermal stability, but experimental melting-temperature (Tm) measurements remain scarce. Molecular simulations can generate stability-related labels far more readily, yet these labels measure quantities other than Tm, and it is unclear which of them actually help an experimental Tm predictor. We trained multi-task models that share an ESM2 sequence representation between a Tm target head and one simulation-derived auxiliary label at a time, in a deliberately low-data setting (57 training, 114 validation, 396 held-out test sequences). To compare very different labels fairly, each was tuned independently and selected only on experimental Tm validation performance, then judged by paired errors on the held-out Tm test set. Two design axes governed transfer. First, data design: a molecular-dynamics native-contact stability label transferred and matched alchemical free-energy perturbation (FEP) when computed on the FEP-matched mutation scans (frozen encoder, paired gain 0.20°C), whereas the same observable over a diverse-nanobody screen did not, because sequence length confounds that screen. Second, the physical observable: the FEP free energy improved Tm with both a frozen and a fine-tuned encoder (best 6.40°C mean absolute error, from a 6.55°C baseline), while the native-contact label helped only a frozen encoder. Rosetta and ThermoMPNN scores were weak or null. Simulation-derived labels can support scarce experimental Tm prediction, but only when a matched data design and a physically transferable observable are combined.

nanobody thermostability | melting-temperature prediction | multi-task transfer learning | protein language model | molecular simulation labels

Correspondence: ***Yasuhiro Matsunaga, ymatsunaga@riken.jp***

Significance

Simulations can label protein variants far faster than experiments, but a simulated label only helps a data-scarce experimental predictor if it actually transfers to the measured phenotype. Using nanobody thermal stability as a testbed, we show that two design choices decide this: matching the simulated variants to the same sequence neighborhood as the target, and choosing a physical observable — an alchemical free energy — that is close enough to stability to reshape a protein language model. This turns "run more simulation" into concrete, testable design rules for building simulation-informed predictors of experimental protein properties.

ORCID: 0000-0003-2872-3908

Introduction

Machine learning can screen molecular designs before experimental testing, but its usefulness is limited by the labels available for training. In materials informatics, this bottleneck has motivated simulation-to-real (Sim2Real) learning, in which models are trained with large computational databases from first-principles calculations, quantum chemistry, or molecular dynamics, then adapted to smaller experimental data sets (1–3). In some polymer and inorganic-material settings, transfer error decreases as the computational database grows, providing a practical test of whether additional simulation is worth generating (3). That test is needed because simulation data are not automatically useful. Model assumptions, finite sampling, and domain gaps can separate a computed label from the experimental property it is meant to support.

Protein engineering has the same label bottleneck, but the relation between computational and experimental labels is often less direct. Experimental measurements are sparse, whereas simulations and structure-based calculations can label many designed or mutated sequences. Those computed labels, however, rarely measure the target phenotype itself. For thermal stability, the experimental target may be the melting temperature (Tm), the assay-dependent temperature at which folded and unfolded populations are balanced. The available computed quantity may instead be a mutation-level free-energy change ($\Delta\Delta G$), a structure-based stability score, or a molecular-dynamics trajectory summary. Such labels are physically related to stability, but they are not interchangeable with Tm. The central mismatch motivates a direct test of which computed quantities transfer to experimental Tm prediction.

We study this question for nanobodies, single-domain VHH antibodies that are small, modular, and useful as engineered binders (4, 5). Their practical use in expression, storage, and formulation depends on thermal stability (6–8), but measuring or simulating that stability directly is costly (9–12). Protein language models (PLMs) such as ESM2 provide a natural starting point because they learn sequence representations from large protein corpora (13–15), and several recent methods use such representations for Tm prediction (16–21). High-throughput stability assays are also changing the field. Tsuboyama et al. measured folding stability for roughly 776,000 natural and designed protein domains by cDNA-display proteolysis (22). Yet those data do not remove the need for nanobody-specific target supervision. The mega-scale domains are 40 to 72 residues long, whereas VHH domains are longer, and models fine-tuned on such data transfer less well to 177 to 501-residue proteins than to small domains (23). For nanobody Tm prediction, target-specific data and target-centered validation remain essential (24–26).

Here, we test whether computed stability labels can supply useful supervision when experimental nanobody Tm labels are scarce. We treat Tm prediction as the target task and each computed label as a computational task that shares an ESM2-based sequence representation (14). The computational labels include mutation free energies from alchemical free-energy perturbation (FEP), Rosetta mutation scores, ThermoMPNN stability scores, Rosetta scores on variants proposed by ESM2 or sampled at random, and an MD-derived native-contact stability label computed either over the same mutation scans used for FEP or, as a control, over a diverse panel of nanobodies. To keep the comparison target-centered and fair across very different labels, every source is tuned independently and selected on experimental Tm validation data, and every final claim is evaluated on the same held-out experimental Tm test set with paired error statistics.

The results are organized by two design axes. First, *data design*: the same MD native-contact observable does not improve Tm as a diverse-nanobody screen but does when it is computed on the FEP-matched mutation scans, where it matches FEP with a frozen encoder — so matching the auxiliary source's sequence design to the target, not the choice of dynamical quantity, is what lets an MD label transfer. Second, the *physical observable*: the alchemical FEP free energy improves Tm whether the encoder is frozen or fine-tuned, whereas the native-contact label helps only a frozen encoder, so the physical content of the label sets how deeply it transfers. The other structure-based scores are weak or null under the same tuning. These results extend the Sim2Real argument to a protein-engineering setting in which the computational and experimental labels differ in physical meaning, and turn "run more simulation" into two concrete design choices — matched data design and a transferable observable — that decide whether computed data help.

Materials and Methods

Experimental and technical design

The study tests whether a computational label can improve experimental nanobody Tm prediction when that label is physically related to Tm but not identical to it. We used a multi-task transfer-learning design in which experimental Tm prediction was the target task, and each computational label was a computational task that shared part of the network with the target. Throughout, model selection and every reported number were tied to experimental Tm performance. Computational-task losses shaped the shared representation during training, but computational-task validation errors were never used to pick checkpoints or hyperparameters.

Experimental Tm data set and split definitions

Experimental Tm labels came from the public nanobody benchmark NbBench, which provides a thermo-seq task with predefined splits (25). We deliberately assigned the smallest partition to training and the largest to the held-out test set, giving 57 training, 114 validation, and 396 test sequences. This places the model in the low-data regime that the study targets. Scarce experimental Tm labels, not abundant ones, are the condition under which simulation-derived supervision is expected to help. The large test set is a direct benefit of this choice, because it makes paired error comparisons and label-count trends easier to resolve.

Split definitions. The training split fit model parameters. The validation split drove checkpoint selection, early stopping, and hyperparameter choice. The test split stayed untouched until the settings were fixed and was used only for final reporting. The same rule held for Tm-only models and for multi-task transfer models.

Target-label scaling. The target CSV files held an amino-acid sequence column and a Tm label in degrees Celsius. During training, Tm labels were mapped to a scaled regression target by robust scaling followed by linear rescaling to [0,1], with the scaler fit only on the 57 training labels. The fitted scaler was then applied to validation labels during training. At evaluation, predictions were inverse-transformed back to degrees Celsius before errors were computed, so the validation and test label distributions never entered the scaling transform.

Experimental-label scaling reference. For this reference, the Tm training split was subsampled to the requested number of labels using the run seed, while the validation and test splits were left intact. These runs give a target-label baseline for judging whether computational-label scaling moves in the expected direction.

Computational-label data sets

Computational labels were loaded as computational tasks, not as additional Tm measurements.

Computational-label classes. The computational labels were mutation free-energy changes from alchemical free-energy perturbation (FEP), structure-based mutation-effect scores from Rosetta, learned mutation-effect scores from ThermoMPNN, Rosetta scores on ESM2-proposed and random variants, and an MD-derived native-contact stability label. The native-contact label was prepared in two data designs: over the FEP-matched 1MEL and 4IDL mutation scans (the primary MD label of this study) and, as a control, over a diverse panel of single-domain antibodies. All computational labels were preprocessed to [0,1]-scaled regression targets and entered the model only through their own computational-task heads.

Processed computational-table sizes. Mutation-effect computational labels were organized as two template-specific tables derived from the 1MEL and 4IDL nanobody structures. FEP, Rosetta, and ThermoMPNN each contained 435 and 409 processed rows for the two templates. The FEP-matched MD native-contact label used the same two scaf-

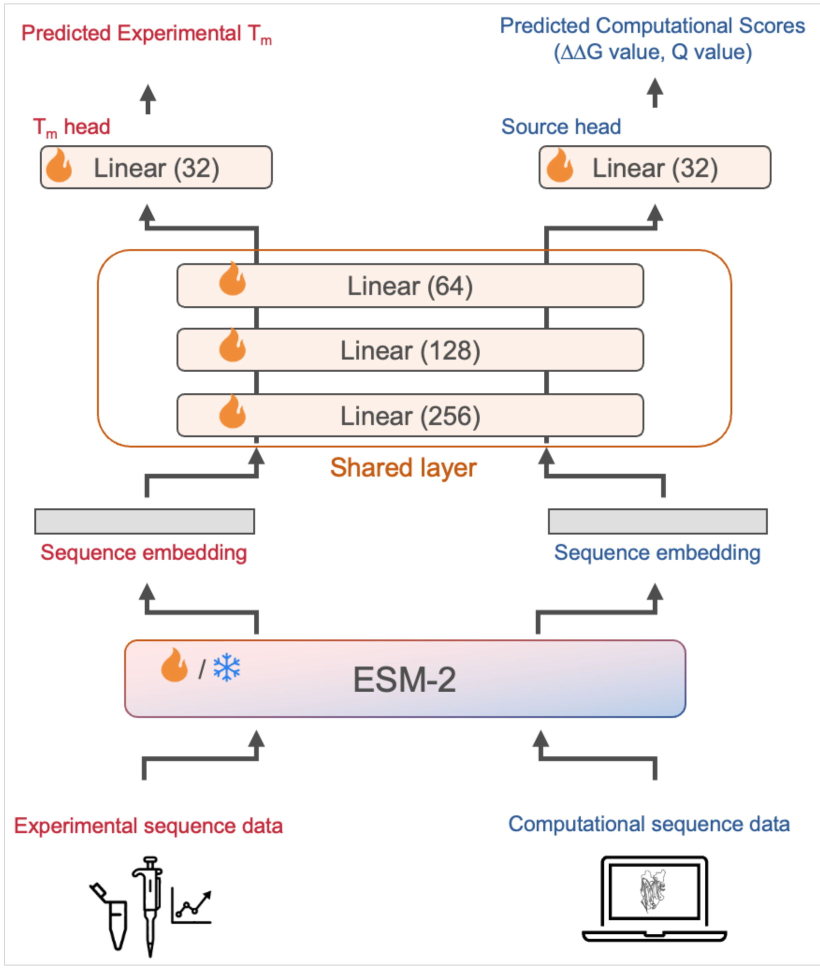

Fig. 1. Multi-task transfer-learning architecture for using simulation data in experimental Tm prediction. Experimental sequence data (left) and computational sequence data (right) are encoded by a shared ESM2 protein language model into per-sequence embeddings. The embeddings pass through a shared multilayer perceptron and then split into two task-specific heads: a Tm head that predicts the experimental melting temperature, and a computational head that predicts the computational scores (mutation $\Delta\Delta G$ values or an MD-derived Q-value). Only the experimental Tm head defines the final objective, while the computational head lets the simulation-derived labels reshape the shared representation during training. Flame and snowflake icons mark trainable and frozen parameters, respectively; the ESM2 encoder is either fine-tuned (hot) or kept frozen depending on the configuration.

folds and contained 431 and 406 processed rows for 1MEL and 4IDL. ESM2-proposed variants and random variants scored by Rosetta each contained 1000 and 1000 processed rows. The diverse-panel MD control came from a separate set of simulated single-domain antibody structures drawn from SAbDab, the Structural Antibody Database (27); its processed table contained 1143 analysis-ready rows. Row counts exclude headers.

Sequence-overlap check. We checked exact sequence overlap between the computational-label tables and the NbBench splits. The mutation-effect tables (FEP, Rosetta, ThermoMPNN, and the FEP-matched MD native-contact table), which are built on the 1MEL and 4IDL scaffolds, and the Rosetta-scored variant tables had no exact match to any NbBench training, validation, or test sequence. The diverse-panel SAbDab MD control table contained a few exact matches, including eight test-set sequences; this table is used only as the negative, length-confounded data-design control and does not enter the positive results, so the overlap cannot account for any reported gain. We report it for transparency.

Computational-label sampling. For a mutation-effect experiment with requested count n, up to n rows were sampled independently from each of the two template tables using the run seed, and the two tables were given separate task identifiers. For an MD-derived Q-value experiment with count n, n rows were sampled from the single Q-value table. Sampled rows were then split 80/20 into computational-training and computational-validation rows with the same seed. The computational-validation rows were kept for monitoring and control analyses, but in the reported protocol the validation data passed to the trainer for checkpoint selection and early stopping were filtered down to the experimental Tm validation rows.

MD native-contact stability labels

The native-contact stability label measures how well a structure's native contacts persist under unbiased molecular dynamics, summarized as a smooth native-contact fraction Q (28). We used it in two data designs.

FEP-matched mutation scans (primary). To match the

FEP label's data design, we ran unbiased MD of the 1MEL and 4IDL wild types and their single-point mutation scans — the same scaffolds and variant lists used for FEP. Each variant was built from the crystal template with the mutation applied, solvated in explicit OPC water with a 15 Å buffer and 0.15 M NaCl, and parameterized with the Amber ff19SB force field and OPC water model with hydrogen-mass repartitioning (29–31). After energy minimization and staged equilibration (NVT then NPT at 300 K), production ran at 400 K in the NVT ensemble with a 4-fs timestep, using OpenMM on CUDA devices (32). For each variant, Q was computed over the first 40 ns of production from backbone-heavy-atom contacts against the variant's own energy-minimized structure, using the same contact definition and averaging window as the control panel (Supplementary Methods). The per-variant label is the wild-type-referenced change $\Delta Q = Q(\text{mutant}) - Q(\text{wild type})$, min–max scaled to [0,1] before training; because a constant wild-type offset is absorbed by this scaling, ΔQ and the raw Q are equivalent network inputs, so any gain over the diverse panel reflects the data design rather than the referencing.

Diverse-panel control. For the data-design comparison we also computed the same native-contact label over a diverse panel of nanobody structures selected from a SAbDab-derived summary table (27). We took one heavy-chain nanobody structure per PDB entry when an IMGT-renumbered coordinate file was available and the chosen chain could be verified. Requiring a usable trajectory, topology, reference structure, a sequence of at least 50 residues, and at least one selected native contact left 1143 trajectories. Each system was prepared from the selected chain alone with PDBFixer and pdb2pqr/PROPKA for pH-aware protonation and Amber-compatible atom naming (33, 34), solvated in OPC water with 15 Å padding and 0.15 M NaCl, and parameterized with ff19SB. Systems were equilibrated at 300 K (1 ns NVT, 1 ns NPT at 1 atm) and a 400 K branch was started through two restrained NVT stages before 100 ns of restraint-free 400 K NVT production with hydrogen-mass repartitioning and a 4-fs timestep. Q was extracted from the 400 K production trajectories with MDTraj (35) and scaled to [0,1]. Because this panel spans a wide range of sequence lengths, its label carries a length confound (Results); it is used only as the negative data-design control.

FEP mutation free-energy labels

The FEP computational labels were mutation free-energy changes for variants of the 1MEL and 4IDL nanobody templates, computed with the same protocol as our related nanobody thermostability study. In brief, alchemical mutation simulations were run with NAMD 3 using GPU acceleration (36). Input systems were prepared in VMD (37) with a customized AlaScan workflow (38), and coordinates were initialized from AlphaFold 3 models of the wild-type sequences (39). Systems used the CHARMM36 protein force field (40), TIP3P water (41), and 150 mM NaCl.

Each transformation used a dual-topology mutation setup with a CHARMM36 hybrid-residue topology, split into 20 equally spaced λ windows from wild type to mutant. Forward and backward transformations were run for 1 ns per direction with a 2-fs timestep and 200 ps of per-window equilibration before data collection, all in the NPT ensemble at 300 K and 1 atm with Langevin temperature control and a Langevin piston barostat. Bonds to hydrogen were constrained with SHAKE (42), and long-range electrostatics used particle mesh Ewald (43). Relative free energies were estimated with the Bennett acceptance ratio (44). The processed tables retain the raw mutation free-energy value, its negated value, and the network-facing [0,1]-scaled label; under this convention, larger processed labels correspond to more favorable mutation free energies.

Rosetta and ThermoMPNN mutation-effect labels

Rosetta mutation-effect labels were generated with the Rosetta `ddg_monomer` application (45). For each template, the PDB structure was cleaned to the target chain, variant sequences were compared with the wild-type sequence, and the differences were written to Rosetta mutation-list files. The calculation used full-atom Rosetta with the `ref2015` score function (46), 50 iterations, Cartesian minimization enabled, local-only optimization disabled, side-chain-minimization-only disabled, minimum-score aggregation, and no PDB dumping. The same protocol scored the template mutation-effect tables, the ESM2-proposed variants, and the random-variant control.

For the ESM2-proposed computational label, ESM2 650M sampled 100,000 two-mutation variants from each wild-type template, ranked them by pseudo-log-likelihood, and kept the top 1% for Rosetta scoring (14). The random-variant control used roughly 1000 two-mutation variants with positions and substitutions chosen at random, scored by the same Rosetta workflow.

ThermoMPNN computational labels were computed from the same template-specific variant sets with ThermoMPNN, a stability-change predictor trained by transfer learning from larger protein-stability data (47), and converted to the same processed CSV format as the other mutation-effect computational labels.

Neural-network architecture

All models used an ESM2 protein language-model encoder (14) with lightweight regression heads. Unless stated otherwise, the encoder was the 8M-parameter ESM2 model. Sequences were tokenized with truncation and padding to 160 residues. The first-token encoder representation fed a shared multilayer perceptron with hidden dimensions 256, 128, and 32, ReLU activations, and dropout. A scalar Tm head predicted the scaled target label; in computational-label transfer models, additional scalar heads predicted the computational labels from the same shared representation.

The Tm-only baseline used the encoder, the shared trunk, and the Tm head with experimental Tm labels alone. Computational-label transfer models kept this target path and added computational heads. The main FEP model used separate computational heads for the two template-specific mutation-effect tasks, the best design in our computational-head comparison. As controls we also tried a shared mutation-effect head, a shared head conditioned on a template embedding, and a shared head with template-specific affine calibration, along with residual and latent computational-fusion variants; the main result used the original shared-trunk architecture.

Hot-encoder and frozen-encoder training

We evaluated two encoder regimes. In the hot-encoder regime, the ESM2 encoder and the regression heads were optimized jointly, with the encoder given a smaller learning rate than the freshly initialized trunk and heads so that the pretrained representation was updated more gently. In the final FEP setting, the selected encoder learning rate was 3×10^{-5} and the head learning rate was the default 3×10^{-4}.

In the frozen-encoder control, all ESM2 parameters were fixed. The encoder was run once to precompute sequence embeddings for the target and computational examples, and training then optimized only the trunk and task heads. This control asks whether the computational labels still help when the pretrained representation is used as a fixed feature extractor. Encoder-size controls repeated selected Tm-only and FEP settings with 35M- and 650M-parameter ESM2 encoders.

Loss function and multi-task objective

Each training example consisted of a sequence x_i, scaled label y_i, and task identifier t_i. The target Tm task had one identifier, and each computational task had its own. A minibatch could mix target and computational examples, but every example contributed loss only through the head for its task.

The per-task regression loss was Huber loss with $\delta = 1.0$ on scaled labels. The default multi-task objective used learnable task-uncertainty weights (48). For task k with loss L_k and learned log-scale s_k, the contribution was

$$\frac{L_k}{2\exp(2s_k)} + s_k,$$

and the total loss summed over the tasks present in the minibatch. Fixed computational-task weights were also evaluated as controls, particularly for the MD-derived labels.

Training, early stopping, and hyperparameter selection

Models were trained with AdamW, cosine learning-rate scheduling, batch size 16, weight decay 0.04, 100 warmup steps, and at most 400 epochs. The default head learning rate was 3×10^{-4}, the default hot-encoder learning rate was 1×10^{-4}, and the default dropout was 0.15. Training used mixed precision on CUDA devices, with checkpoints saved and evaluated once per epoch.

Within each run, the best checkpoint was the one with the lowest mean squared error on the experimental Tm validation examples, and early stopping used the same metric with a patience of 15 epochs. Computational-label rows were present in the training set, but they were removed from the validation data passed to the trainer for checkpoint selection and early stopping. This is the implementation-level step that keeps reported checkpoints selected by target-task validation performance rather than by computational-task performance.

Hyperparameters were selected on the experimental Tm validation split, and to keep the comparison fair we tuned every source independently and separately for the frozen and hot encoder regimes, using a two-stage search. Stage 1 fixed the learning rate, dropout, and weight decay at defaults and searched the model architecture (shared trunk, residual fusion, or latent fusion) crossed with the auxiliary-head coupling (separate template-specific heads or a template-conditioned shared head). Stage 2 held the best Stage 1 skeleton and searched, for the hot regime, the encoder learning rate (1×10^{-4}, 5×10^{-5}, 3×10^{-5}, 1×10^{-5}) and, for the frozen regime, the head learning rate (1×10^{-3}, 5×10^{-4}, 2×10^{-4}), each crossed with dropout (0.05, 0.15, 0.30) and weight decay (0.02, 0.10). Each candidate was evaluated as a three-seed ensemble on the Tm validation split at 320 auxiliary labels, and the single best configuration for each source and regime was then evaluated on the held-out test split.

Independent run seeds controlled model initialization, target-training subsampling in the target-label scaling experiment, computational-label sampling, and computational-task train/validation splitting. Final test evaluations used five-seed ensembles. The label-count curves used the selected per-source configuration while varying the number of computational labels (20, 80, 160, 320), rather than retuning a separate model at every label-count point.

Final evaluation and statistics

Final evaluation used the same 396 held-out Tm test examples for every condition. For each condition, five independently seeded models produced scaled Tm predictions for every test sequence; these were averaged in scaled-label space and inverse-transformed to degrees Celsius. The primary metric was mean absolute error (MAE) in degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations were also computed, but MAE anchored the main comparisons because it reads directly as an average temperature error.

Confidence intervals came from nonparametric bootstrap resampling over held-out test proteins. For a single condition, the absolute-error vector from the five-seed ensemble was resampled with replacement 10,000 times using a fixed random seed, and the reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. We report 90% intervals because the comparisons are directional, paired, and intended to emphasize effect sizes rather than binary significance thresholds; the bootstrap summaries retained by the analysis code allow other interval levels to be recomputed from the same resamples.

For paired comparisons, the same bootstrap indices were applied to the absolute-error vectors of the candidate and reference conditions, and the candidate-minus-reference MAE difference was computed for each bootstrap sample. This paired bootstrap preserves the matched-test-example structure of the comparison. Throughout, ΔMAE is candidate minus reference, so negative values mean lower error than the reference. The evaluation code also reports the one-sided tail probability, defined as the fraction of paired bootstrap samples with ΔMAE above zero, but we emphasize effect sizes and 90% confidence intervals.

Implementation details

The pipeline builds the target and computational tasks, applies target-label scaling, performs seeded computational-label sampling, filters validation data for target-task checkpoint selection, evaluates seed ensembles, and writes bootstrap statistics. Each run emits a machine-readable summary with the command-line arguments, resolved model settings, per-point metrics, bootstrap intervals, and paired comparisons. These summaries feed the figure-generation workflow directly and will be archived with the processed data for reproducibility.

Results

A target-centered framework for using simulation labels in Tm prediction

The target task was nanobody melting-temperature (Tm) regression from amino-acid sequence. Computational labels were treated as auxiliary prediction tasks, not as stand-ins for experimental Tm. Each model used the pretrained protein sequence model ESM2 to turn a sequence into a learned representation that a Tm head and an auxiliary head shared, so the auxiliary task could reshape the representation while the final objective stayed anchored to experimental Tm (Fig. 1).

We used the benchmark split as a deliberately low-data target setting, with 57 experimental Tm labels for training, 114 for validation, and 396 for held-out testing. To keep the comparison fair across very different auxiliary labels, each source was tuned independently: for every source and every encoder regime we searched the model architecture, the auxiliary-head coupling, and the learning rate, dropout, and weight decay, selecting on the experimental Tm validation set alone. Only the selected configuration was then evaluated on the held-out Tm test set, and improvements were measured as paired differences in absolute error on the same test examples. This target-centered protocol guards against a trap specific to simulation-to-experiment learning: an auxiliary task can be learnable and physically sensible yet do nothing for the experimental phenotype. Under this protocol the tuned Tm-only baselines were 7.23°C MAE with a frozen encoder and 6.55°C MAE with a fine-tuned (hot) encoder. Two design axes organize the results: whether a simulation label transfers at all is set by its *data design* (Fig. 2), and how deeply it transfers is set by its *physical observable* (Fig. 3).

Data design: matching the sequence scan makes an MD label transfer

With each source tuned to the same Tm endpoint, two all-atom labels clearly transferred: the FEP mutation free energy and — new to this work — an MD native-contact stability label computed on the *same* 1MEL and 4IDL mutation scans used for FEP. The MD native-contact label uses one physical observable in two different data designs, which isolates the effect of data design from the choice of quantity. Computed over an earlier screen of diverse nanobodies, the label did not improve Tm: its held-out error sat at or slightly above the corresponding Tm-only reference (Fig. 2a, upper group). Computed instead over the FEP-matched mutation scans — the same scaffolds, all of constant length — the identical observable improved Tm and matched FEP with a frozen encoder (Fig. 2a, lower group). Because the two groups come from different training series with different absolute baselines, each is shown against its own Tm-only reference and the two are not cross-compared numerically.

The difference is a data-design confound, not the label arithmetic (Fig. 2b). The diverse screen spans sequence lengths from 58 to 461 residues and carries a nonzero label–length correlation ($r = +0.13$), a length signal the auxiliary task can latch onto instead of the stability signal; the matched scans are of constant length, so no such confound exists. Referencing each mutant's native-contact value to its wild type (a ΔQ) versus using the raw value is immaterial here, because a constant offset is absorbed by the min–max scaling applied to every label. With the confound removed, both all-atom labels scale below the Tm-only baseline as the number of auxiliary labels grows: with a frozen encoder the FEP curve stays below the baseline throughout (7.13 at 20 labels to 7.01 at 320), and the matched-scan MD curve falls from above it to below it, crossing by 320 labels (7.32 to 7.03; Fig. 2c). At 320 labels the matched MD label improved held-out Tm by -0.20°C (90% CI -0.34 to -0.05), statistically indistinguishable from FEP in the same regime (-0.22°C; paired FEP-minus-MD difference -0.03°C, 90% CI -0.21 to $+0.16$). Matching the auxiliary source's sequence design to the target neighborhood, not the specific dynamical quantity, is what lets the MD label transfer.

Physical observable: free energy transfers deeper than native contacts

Tuned to the same endpoint, the sources formed a clear frozen-encoder hierarchy (Fig. 3a): FEP (7.01) and the matched-scan MD label (7.03) led, both improving on the Tm-only baseline (7.23); ThermoMPNN was a weak third (7.09, 90% CI crossing zero); and the Rosetta-family scores — the plain Rosetta mutation score and Rosetta scores on ESM2-proposed or random variant sets — sat at the baseline (7.23, 7.31, and 7.22). But a matched data design was necessary, not sufficient: the two all-atom labels differed in how deeply they transferred across encoder regimes (Fig. 3b). The FEP free energy improved Tm whether the encoder was frozen or fine-tuned (frozen -0.22°C, 90% CI -0.36 to -0.08; hot -0.15°C, 90% CI -0.30 to -0.01), reaching the best held-out accuracy overall (6.40°C MAE, hot). The MD native-contact label improved Tm only with a frozen encoder; once the encoder was fine-tuned it gave no net gain and, at small label counts, degraded prediction (paired $+0.47$°C at 20 labels, 90% CI $+0.29$ to $+0.65$), recovering only to the

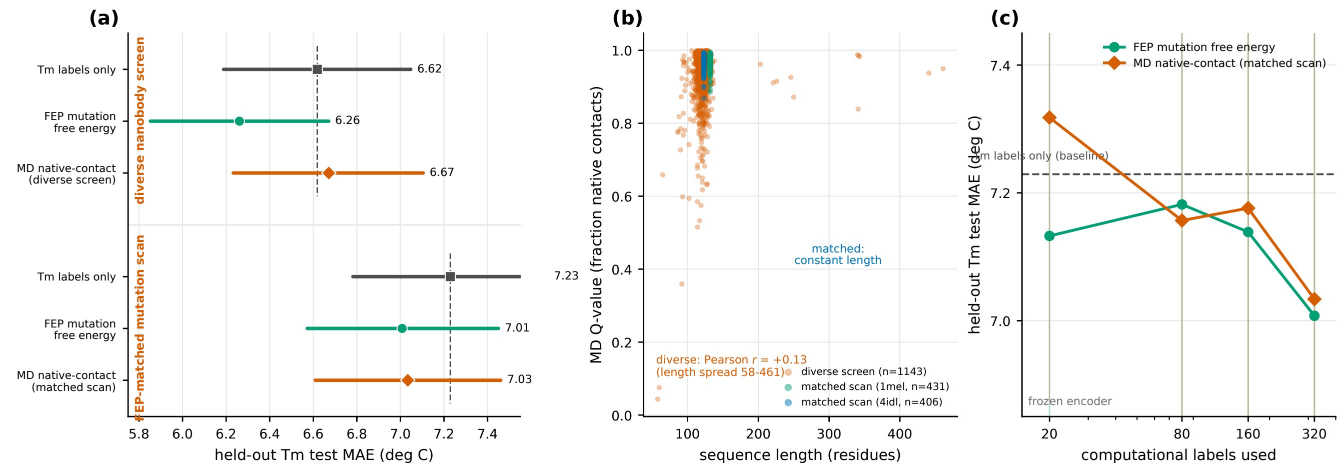

Fig. 2. Data-design axis: a matched mutation scan makes the MD label transfer. (a) Held-out test MAE for the MD native-contact label computed over a diverse nanobody screen (upper) versus the FEP-matched 1MEL/4IDL mutation scan (lower), each with its own Tm-only and FEP references. The two groups are from different training series (different absolute baselines) and are not compared numerically across groups; the point is null-versus-positive for the same observable. (b) The confound made explicit: MD native-contact value versus sequence length. The diverse screen has a broad length spread (58–461 residues) and a nonzero label–length correlation ($r = +0.13$); the matched scans are constant length. (c) Frozen-encoder label-count sweeps for FEP and the FEP-matched MD native-contact label; the dashed line is the Tm-only frozen baseline. Both labels fall below the baseline as labels increase, and the matched MD label crosses it by 320 labels. Points and intervals show ensemble MAE and 90% bootstrap intervals.

baseline by 320 labels. Directly, FEP beat the MD label in the hot regime (-0.18°C paired, 90% CI -0.36 to -0.01) while tying it when frozen.

The natural reading is that the alchemical free energy carries enough information about the mutation's effect on stability to beneficially reshape the sequence encoder, whereas a native-contact persistence label can usefully condition a fixed representation but cannot steer encoder fine-tuning toward the Tm phenotype (Fig. 3c). The physical content of the label sets the *depth* of transfer. The Rosetta-family scores fit the weaker side of this picture: they did not robustly improve Tm in either regime. In particular the two Rosetta-scored variant sets (ESM2-proposed and random variants) are ordinary comparators; under tuning the ESM2-proposed set did not outperform the random set, so we make no claim about physics-scored sequence proposals as a design loop. Enlarging the ESM2 encoder from 8M to 35M or 650M parameters did not by itself reproduce the FEP gain (Supplementary Fig. 4), so model capacity does not substitute for a matched physical label.

Across every label we tested, the improvements were selective rather than additive. Two axes organize the pattern: a matched data design decides whether a simulation label transfers at all, and the physical observable decides how deep that transfer goes. Under both, only the FEP mutation free energy (in both regimes) and the FEP-matched MD native-contact label (with a frozen encoder) robustly improved low-data nanobody Tm prediction.

Discussion

This study asked how simulation-derived stability labels should be used when the experimental target is scarce and the simulated quantity is related to, but not identical to, that target. In low-data nanobody Tm prediction the answer separated into two design axes. Matching the auxiliary source's sequence design to the target decided whether a molecular-dynamics label transferred at all, and the physical observable decided how deeply it transferred. Under matched per-source tuning, only the FEP mutation free energy (with a frozen or a fine-tuned encoder) and the FEP-matched MD native-contact label (with a frozen encoder) robustly improved held-out Tm, while the Rosetta-family scores were weak or null. The absolute gains were small in degrees Celsius, so they should be read as statistically consistent improvements in a low-data predictor rather than as assay-level stability changes of the same magnitude.

Data design decides whether an MD label transfers. The clearest new result is that the same MD native-contact observable can be null or useful depending only on how its variants are chosen. Computed over a diverse panel of nanobodies the label did not improve Tm; computed over the FEP-matched 1MEL and 4IDL mutation scans it improved Tm and matched FEP with a frozen encoder. The diverse panel spans a wide range of sequence lengths and carries a nonzero label–length correlation, a confound the auxiliary head can fit instead of the stability signal; the matched scans hold length constant and present a clean mutation-to-stability gradient. Because a constant wild-type offset is absorbed by the min–max scaling, referencing each mutant to its wild type (ΔQ) is numerically equivalent to using the raw contact fraction, so the gain is attributable to the matched sequence design and not to the label arithmetic. This reframes an earlier apparent negative result — that an MD-derived stability label does not help — as a statement about data design rather than about molecular dynamics.

The physical observable sets the depth of transfer. With data design matched, FEP and the native-contact label still behaved differently across encoder regimes. FEP improved Tm whether the encoder was frozen or fine-tuned, whereas the native-contact label helped only when the encoder was

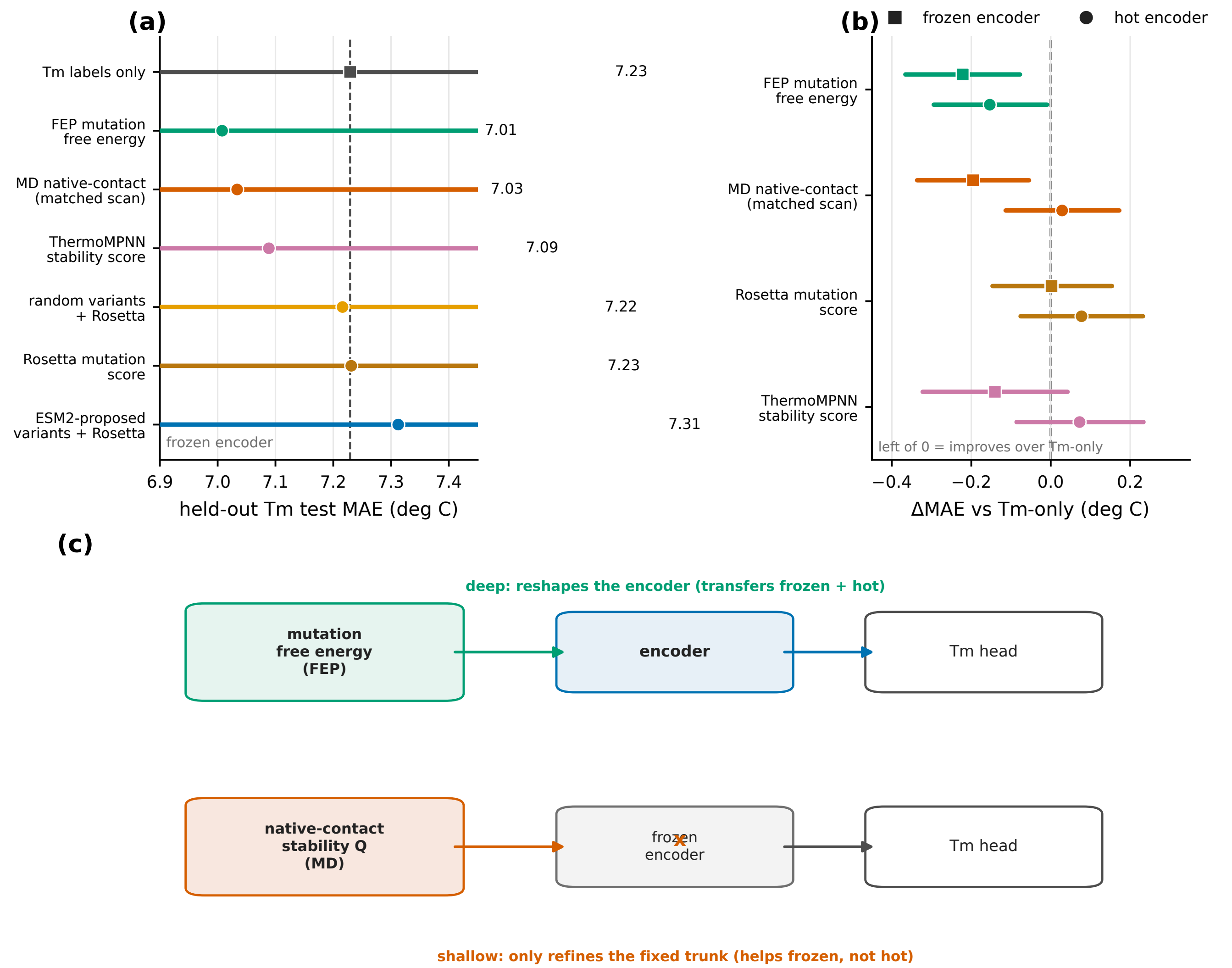

Fig. 3. Physical-observable axis: depth of transfer set by the simulated quantity. (a) Tuned frozen-encoder held-out test MAE for every source; the dashed line is the Tm-only baseline. FEP and the matched-scan MD label lead; the Rosetta-family scores sit at the baseline. (b) Paired ΔMAE versus Tm-only for FEP, the MD native-contact label, Rosetta, and ThermoMPNN under frozen versus hot encoders; values left of zero improve on Tm-only. FEP crosses below zero in both regimes; the MD native-contact label only when frozen; Rosetta and ThermoMPNN sit near zero. (c) Interpretation: the free-energy label reshapes the shared encoder (deep; transfers frozen and hot), whereas the native-contact label only refines the fixed trunk (shallow; helps frozen, not hot). Points and intervals show ensemble MAE and 90% bootstrap intervals.

frozen and gave no net gain, or a small degradation at low label counts, once the encoder was fine-tuned. A frozen encoder lets an auxiliary label recombine fixed features through the shared trunk; fine-tuning additionally lets it reshape the sequence representation itself. The alchemical free energy carries enough mutation-specific stability information to make that reshaping beneficial, while a native-contact persistence statistic can usefully condition a fixed representation but does not have the leverage to steer encoder fine-tuning toward the Tm phenotype. The physical content of the label thus sets not just whether it helps but how deeply it can act.

Why the free-energy label works. FEP estimates how a mutation changes folding free energy, a relative $\Delta\Delta G$, whereas Tm is the absolute temperature at which folded and unfolded populations balance. The two are not interchangeable, but they read out related aspects of the same stability landscape: sparse Tm measurements anchor the absolute scale, and mutation free energies supply local directions in sequence space. That shared physical content was enough for $\Delta\Delta G$ supervision, selected only by Tm validation, to improve held-out Tm even though the auxiliary variants differ from the Tm test sequences. This interpretation also separates label content from generic auxiliary-task regularization, because a native-contact task supplied the same kind of extra signal yet only helped in the shallower (frozen) regime.

Controls locate the signal. Three controls sharpen the reading. First, model size was not the lever: 35M and 650M ESM2 encoders did not beat the 8M encoder in either the Tm-only or FEP settings, consistent with the observation that fine-tuning rather than raw capacity drives PLM-based property prediction (26). Second, the frozen-versus-hot contrast is exactly the physical-observable axis above and shows the effect is about representational depth, not optimizer settings, because each source was retuned within each regime. Third, a net-charge finite-size correction for the buried-charge FEP mutations was negligible for the scaled labels, so it does not affect the comparison. We also kept the extreme FEP values rather than clipping them, since the extreme buried-charge mutations — experimentally hard to measure — carry a disproportionate share of the transferable signal.

Selectivity and what we do not claim. The improvements were selective, not additive. The Rosetta-family scores — the plain Rosetta mutation score, ThermoMPNN, and Rosetta scores on ESM2-proposed or random variant sets — did not robustly improve Tm under matched tuning, with ThermoMPNN giving only a weak, non-significant frozen gain. In particular, the Rosetta scores on ESM2-proposed variants did not outperform the random-variant control once each was tuned, so we make no claim that language-model proposals scored by physics constitute a useful design loop; those two sources appear only as ordinary comparators. Establishing a generator–physics–predictor design loop, and any active- or reinforcement-learning extension, is outside the scope of this supervised study.

Connection to Sim2Real learning and cost. These results extend simulation-to-real learning (3) to a protein-engineering setting where the computational and experimental labels differ in physical meaning. Materials-informatics studies show that transferred error can fall as a related computational database grows; here the analogous quantity is a matched-design $\Delta\Delta G$ pool, which our two-template FEP set only begins to sample. Because that pool was small and the label-count curves were not strictly monotonic, we do not claim a clean scaling exponent, and an equivalent-sample-size estimate (the number of simulation labels that substitute for one experimental label) is only planning-scale here. Cost trades off against transfer strength: learned or structure-based scores label variants far faster than all-atom alchemical calculations but gave weak-to-null transfer, whereas FEP was the most expensive per mutation yet the strongest per label. A tiered strategy — cheap scores for breadth, all-atom calculations where a directly stability-related label is worth the cost — follows naturally, though we did not measure cost systematically.

Limitations

Several limitations bound the interpretation. The experimental Tm training set is small (57 sequences), so the conclusions should be retested on other nanobody collections and protein families. The auxiliary-label comparison covers only the labels available here; other free-energy protocols, stability predictors, or trajectory observables may transfer differently. The FEP and matched-MD label-count curves support beneficial trends at the largest tested setting but do not establish a clean scaling exponent, and the bootstrap intervals are wide, so the source rankings are best read as robust patterns rather than precise gaps. The FEP-matched labels are built on only two templates; broader template and mutation coverage is the natural next dataset. The diverse-panel MD control contained a few exact NbBench sequence overlaps, but it is used only as the negative data-design control and does not enter the positive results. No new experimental measurements were made to validate high-stability candidates, and because raw trajectories may be too large to deposit in full, reproducibility relies on processed data, scripts, and a clearly reported protocol.

Conclusion

Simulation-derived labels can improve low-data nanobody Tm prediction, but only under two design conditions that this study makes explicit. The auxiliary source must share the sequence design of the target, or a length confound cancels its signal; and its physical observable must be close enough to stability to reshape the sequence representation, or it helps only when that representation is held fixed. Under both conditions, alchemical free energies transferred in every encoder regime and an FEP-matched molecular-dynamics native-contact label matched them with a frozen encoder, while structure-based scores were weak or null. These two axes — matched data design and a transferable physical observable — turn "run more simulation" into concrete, testable choices for building simulation-informed predictors of scarce experimental protein properties, and they define the datasets worth collecting next: matched-design mutation free energies at scale, paired with a modest, well-distributed set of experimental Tm anchors.

Conflict of interest

Taihei Murakami is an employee of Epsilon Molecular Engineering, Inc. Yasuhiro Matsunaga is a scientific advisor of Epsilon Molecular Engineering, Inc. The other authors declare no competing interests.

Author contributions

Taihei Murakami and Kentaro Sasaki contributed equally to this work. Yasuhiro Matsunaga conceived and supervised the study, acquired funding, developed the analysis and training code, performed the analyses, interpreted the results, prepared the figures, and wrote the manuscript. Kentaro Sasaki performed the computational free-energy and structure-based calculations. Soichiro Oda and Kazuma Okada performed molecular-dynamics simulations and data curation. Taihei Murakami contributed to draft writing. All authors reviewed and approved the final manuscript.

Data availability

Processed data required to reproduce the main and supplementary figures are available on GitHub (see below) and will be deposited in Zenodo upon acceptance (DOI to be assigned). Raw simulation trajectories are large; solvent-stripped, frame-thinned trajectories sufficient to reproduce the published labels will be deposited in the same Zenodo record, and the full raw data are available upon request. Analysis, training, and figure-generation code is available on GitHub at `https://github.com/ymatsunaga/`

sim2real-nanobody-tm; an archival release will be deposited in Zenodo upon acceptance.

Acknowledgements

This work was supported by JSPS KAKENHI Grant 23H03412 and JST NBDC Grant JPMJND2603, and partly supported by MEXT as "Program for Promoting Researches on the Supercomputer Fugaku" (Development and application of large-scale simulation-based inferences for biomolecules JPMXP1020230119). Computational resources were provided by the supercomputer Fugaku at the RIKEN Center for Computational Science under Project IDs hp230209, hp240215, and hp250233. AI-assisted tools were used to support language editing, code development, and figure preparation; the authors reviewed and take responsibility for all manuscript content, analyses, and conclusions.

References

1. Hironao Yamada, Chang Liu, Stephen Wu, Yukinori Koyama, Shenghong Ju, Junichiro Shiomi, Junko Morikawa, and Ryo Yoshida. Predicting materials properties with little data using shotgun transfer learning. *ACS Central Science*, 5(10):1717–1730, 2019. doi: 10.1021/acscentsci.9b00804.
2. Yuta Aoki, Stephen Wu, Teruki Tsurimoto, Yoshihiro Hayashi, Shunya Minami, Tadamichi Okubo, Kazuya Shiratori, and Ryo Yoshida. Multitask machine learning to predict polymer–solvent miscibility using Flory–Huggins interaction parameters. *Macromolecules*, 56(14): 5446–5456, 2023. doi: 10.1021/acs.macromol.2c02600.
3. Shunya Minami, Yoshihiro Hayashi, Stephen Wu, Kenji Fukumizu, Hiroki Sugisawa, Masashi Ishii, Isao Kuwajima, Kazuya Shiratori, and Ryo Yoshida. Scaling law of Sim2Real transfer learning in expanding computational materials databases for real-world predictions. *npj Computational Materials*, 11:146, 2025. doi: 10.1038/s41524-025-01606-5.
4. Serge Muyldermans. Nanobodies: Natural single-domain antibodies. *Annual Review of Biochemistry*, 82:775–797, 2013. doi: 10.1146/annurev-biochem-063011-092449.
5. Elena Alexander and Kam W. Leong. Discovery of nanobodies: a comprehensive review of their applications and potential over the past five years. *Journal of Nanobiotechnology*, 22 (1):661, 2024. doi: 10.1186/s12951-024-02900-y.
6. Patrick Kunz, Katinka Zinner, Norbert Mücke, Tanja Bartoschik, Serge Muyldermans, and Jörg D Hoheisel. The structural basis of nanobody unfolding reversibility and thermoresistance. *Sci. Rep.*, 8(1):7934, 2018.
7. Tara M Mezzasalma, James K Kranz, Winnie Chan, Geoffrey T Struble, Céline Schalk-Hihi, Ingrid C Deckman, Barry A Springer, and Matthew J Todd. Enhancing recombinant protein quality and yield by protein stability profiling. *J. Biomol. Screen.*, 12(3):418–428, 2007.
8. S Kumar, C J Tsai, and R Nussinov. Factors enhancing protein thermostability. *Protein Eng.*, 13(3):179–191, 2000.
9. Gert-Jan Bekker, Benson Ma, and Narutoshi Kamiya. Thermal stability of single-domain antibodies estimated by molecular dynamics simulations. *Protein Science*, 28(2):429–438, 2019. doi: 10.1002/pro.3546.
10. Ren Higashida and Yasuhiro Matsunaga. Enhanced conformational sampling of nanobody CDR H3 loop by generalized replica-exchange with solute tempering. *Life (Basel)*, 11(12): 1428, 2021.
11. Ammar Mohseni, Maryam Molakarimi, Majid Taghdir, Reza H Sajedi, and Sadegh Hasannia. Exploring single-domain antibody thermostability by molecular dynamics simulation. *J. Biomol. Struct. Dyn.*, 37(14):3686–3696, 2019.
12. Yusuke Tomimoto, Rika Yamazaki, and Hiroki Shirai. Increasing the melting temperature of VHH with the in silico free energy score. *Sci. Rep.*, 13(1):4922, 2023.
13. Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences of the United States of America*, 118(15): e2016239118, 2021. doi: 10.1073/pnas.2016239118.
14. Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
15. Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S Molina, Neil Thomas, Yousuf A Khan, Chetan Mishra, Carolyn Kim, Liam J Bartie, Matthew Nemeth, Patrick D Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025.
16. Lasse M. Blaabjerg, Maher M. Kassem, Liam L. Good, Nicolas Jonsson, Matteo Cagiada, Kristoffer E. Johansson, Wouter Boomsma, Amelie Stein, and Kresten Lindorff-Larsen. Rapid protein stability prediction using deep learning representations. *eLife*, 12:e82593, 2023. doi: 10.7554/eLife.82593.
17. Mengyu Li, Hongzhao Wang, Zhenwu Yang, Longgui Zhang, and Yushan Zhu. DeepTM: A deep learning algorithm for prediction of melting temperature of thermophilic proteins directly from sequences. *Computational and Structural Biotechnology Journal*, 21:5544–5560, 2023. doi: 10.1016/j.csbj.2023.11.006.
18. J. A. E. Alvarez and S. N. Dean. TEMPRO: nanobody melting temperature estimation model using protein embeddings. *Scientific Reports*, 14:19074, 2024. doi: 10.1038/s41598-024-70101-6.
19. Castrense Savojardo, Matteo Manfredi, Pier Luigi Martelli, and Rita Casadio. DDGemb: predicting protein stability change upon single- and multi-point variations with embeddings and deep learning. *Bioinformatics*, 41(1), 2024.
20. Aubin Ramon, Mingyang Ni, Olga Predeina, Rebecca Gaffey, Patrick Kunz, Shimobi Onuoha, and Pietro Sormanni. Prediction of protein biophysical traits from limited data: a case study on nanobody thermostability through NanoMelt. *MAbs*, 17(1):2442750, 2025.
21. Jun Mao, Yuanpeng Song, Ming Kong, Yanzhi Guo, Yijing Liu, and Xuemei Pu. Thermostability prediction powered by synergistic deep learning at experimental and theoretical levels for nanobodies. *ACS Appl. Mater. Interfaces*, 18(4):6433–6451, 2026.
22. Kotaro Tsuboyama, Justas Dauparas, Jonathan Chen, Elodie Laine, Yasser Mohseni Behbahani, Jonathan J. Weinstein, Niall M. Mangan, Sergey Ovchinnikov, and Gabriel J. Rocklin. Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973):434–444, 2023. doi: 10.1038/s41586-023-06328-6.
23. Simon K. S. Chu, Kush Narang, and Justin B. Siegel. Protein stability prediction by fine-tuning a protein language model on a mega-scale dataset. *PLOS Computational Biology*, 20(7):e1012248, 2024. doi: 10.1371/journal.pcbi.1012248.
24. Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023: baad021, 2023. doi: 10.1093/database/baad021.
25. Yiming Zhang and Koji Tsuda. NbBench: Benchmarking language models for comprehensive nanobody tasks, 2025.
26. Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15:7407, 2024. doi: 10.1038/s41467-024-51844-2.
27. James Dunbar, Konrad Krawczyk, Jinwoo Leem, Terry Baker, Angelika Fuchs, Guy Georges, Jiye Shi, and Charlotte M. Deane. SAbDab: the structural antibody database. *Nucleic Acids Research*, 42(D1):D1140–D1146, 2014. doi: 10.1093/nar/gkt1043.
28. Robert B. Best, Gerhard Hummer, and William A. Eaton. Native contacts determine protein folding mechanisms in atomistic simulations. *Proceedings of the National Academy of Sciences of the United States of America*, 110(44):17874–17879, 2013. doi: 10.1073/pnas.1311599110.
29. David A. Case, Hasan Metin Aktulga, Kellon Belfon, David S. Cerutti, G. Andres Cisneros, Vinicius W. D. Cruzeiro, Negin Forouzesh, Timothy J. Giese, Andreas W. Götz, Holger Gohlke, Saeed Izadi, Koushik Kasavajhala, Mehmet C. Kaymak, Edward King, Tom Kurtzman, Tai-Sung Lee, Pengfei Li, Jian Liu, Tyler Luchko, Ray Luo, Madushanka Manathunga, Matias R. Machado, Hai Minh Nguyen, Kurt A. O'Hearn, Alexey V. Onufriev, Feng Pan, Sergio Pantano, Ruxi Qi, Ali Rahnamoun, Ali Risheh, Stephan Schott-Verdugo, Akhil Shajan, Jason M. Swails, Junmei Wang, Haixin Wei, Xiongwu Wu, Yongxian Wu, Shi Zhang, Shiji Zhao, Qiang Zhu, Thomas E. Cheatham, Daniel R. Roe, Adrian E. Roitberg, Carlos Simmerling, Darrin M. York, Maria C. Nagan, and Kenneth M. Merz. AmberTools. *Journal of Chemical Information and Modeling*, 63(20):6183–6191, 2023. doi: 10.1021/acs.jcim.3c01153.
30. Chuan Tian, Koushik Kasavajhala, Kellon A. A. Belfon, Lauren Raguette, He Huang, Angela N. Migues, John Bickel, Yuzhang Wang, Jorge Pincay, Qin Wu, and Carlos Simmerling. ff19SB: amino-acid-specific protein backbone parameters trained against quantum mechanics energy surfaces in solution. *Journal of Chemical Theory and Computation*, 16 (1):528–552, 2020. doi: 10.1021/acs.jctc.9b00591.
31. Saeed Izadi, Ramu Anandakrishnan, and Alexey V. Onufriev. Building water models: a different approach. *The Journal of Physical Chemistry Letters*, 5(21):3863–3871, 2014. doi: 10.1021/jz501780a.
32. Peter Eastman, Jason Swails, John D. Chodera, Robert T. McGibbon, Yutong Zhao, Kyle A. Beauchamp, Lee-Ping Wang, Andrew C. Simmonett, Matthew P. Harrigan, Chaya D. Stern, Rafal P. Wiewiora, Bernard R. Brooks, and Vijay S. Pande. OpenMM 7: rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*, 13 (7):e1005659, 2017. doi: 10.1371/journal.pcbi.1005659.
33. Todd J. Dolinsky, Jens E. Nielsen, J. Andrew McCammon, and Nathan A. Baker. PDB2PQR: an automated pipeline for the setup of Poisson–Boltzmann electrostatics calculations. *Nucleic Acids Research*, 32(Web Server issue):W665–W667, 2004. doi: 10.1093/nar/gkh381.
34. Mats H. M. Olsson, Chresten R. Søndergaard, Michal Rostkowski, and Jan H. Jensen. PROPKA3: consistent treatment of internal and surface residues in empirical pK_a predictions. *Journal of Chemical Theory and Computation*, 7(2):525–537, 2011. doi: 10.1021/ct100578z.
35. Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. MDTraj: a modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.
36. James C. Phillips, David J. Hardy, Julio D. C. Maia, John E. Stone, João V. Ribeiro, Rafael C. Bernardi, Ronak Buch, Giacomo Fiorin, Jérôme Hénin, Wei Jiang, Ryan McGreevy, Marcelo C. R. Melo, Brian K. Radak, Robert D. Skeel, Abhishek Singharoy, Yi Wang, Benoît Roux, Aleksei Aksimentiev, Zaida Luthey-Schulten, Laxmikant V. Kalé, Klaus Schulten, Christophe Chipot, and Emad Tajkhorshid. Scalable molecular dynamics on CPU and GPU architectures with NAMD. *The Journal of Chemical Physics*, 153(4):044130, 2020. doi: 10.1063/5.0014475.
37. William Humphrey, Andrew Dalke, and Klaus Schulten. VMD: Visual molecular dynamics. *Journal of Molecular Graphics*, 14(1):33–38, 1996. doi: 10.1016/0263-7855(96)00018-5.
38. Vijayaraj Ramadoss, François Dehez, and Christophe Chipot. AlaScan: A graphical user interface for alanine scanning free-energy calculations. *Journal of Chemical Information and Modeling*, 56(6):1122–1126, 2016. doi: 10.1021/acs.jcim.6b00162.
39. Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, Sebastian W.

Bodenstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Akvilė Žemgulytė, Eirini Arvaniti, Charles Beattie, Ottavia Bertolli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousuf A. Khan, Caroline M. R. Low, Kuba Perlin, Anna Potapenko, Pascal Savy, Sukhdeep Singh, Adrian Stecula, Ashok Thillaisundaram, Catherine Tong, Sergei Yakneen, Ellen D. Zhong, Michal Zielinski, Augustin Žídek, Victor Bapst, Pushmeet Kohli, Max Jaderberg, Demis Hassabis, and John M. Jumper. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.

40. Robert B. Best, Xiao Zhu, Jihyun Shim, Pedro E. M. Lopes, Jeetain Mittal, Michael Feig, and Alexander D. MacKerell. Optimization of the additive CHARMM all-atom protein force field targeting improved sampling of the backbone ϕ, ψ and side-chain χ_1 and χ_2 dihedral angles. *Journal of Chemical Theory and Computation*, 8(9):3257–3273, 2012. doi: 10.1021/ct300400x.
41. William L. Jorgensen, Jayaraman Chandrasekhar, Jeffry D. Madura, Roger W. Impey, and Michael L. Klein. Comparison of simple potential functions for simulating liquid water. *The Journal of Chemical Physics*, 79(2):926–935, 1983. doi: 10.1063/1.445869.
42. Jean-Paul Ryckaert, Giovanni Ciccotti, and Herman J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. *Journal of Computational Physics*, 23(3):327–341, 1977. doi: 10.1016/0021-9991(77)90098-5.
43. Ulrich Essmann, Lalith Perera, Max L. Berkowitz, Tom Darden, Hsing Lee, and Lee G. Pedersen. A smooth particle mesh Ewald method. *The Journal of Chemical Physics*, 103 (19):8577–8593, 1995. doi: 10.1063/1.470117.
44. Charles H. Bennett. Efficient estimation of free energy differences from Monte Carlo data. *Journal of Computational Physics*, 22(2):245–268, 1976. doi: 10.1016/0021-9991(76)90078-4.
45. Elizabeth H. Kellogg, Andrew Leaver-Fay, and David Baker. Role of conformational sampling in computing mutation-induced changes in protein structure and stability. *Proteins: Structure, Function, and Bioinformatics*, 79(3):830–838, 2011. doi: 10.1002/prot.22921.
46. Rebecca F. Alford, Andrew Leaver-Fay, Jeliazko R. Jeliazkov, Matthew J. O'Meara, Frank P. DiMaio, Hahnbeom Park, Maxim V. Shapovalov, P. Douglas Renfrew, Vikram K. Mulligan, Kalli Kappel, Jason W. Labonte, Michael S. Pacella, Richard Bonneau, Philip Bradley, Roland L. Dunbrack, Rhiju Das, David Baker, Brian Kuhlman, Tanja Kortemme, and Jeffrey J. Gray. The Rosetta all-atom energy function for macromolecular modeling and design. *Journal of Chemical Theory and Computation*, 13(6):3031–3048, 2017. doi: 10.1021/acs.jctc.7b00125.
47. Henry Dieckhaus, Michael Brocidiacono, Nicholas Z. Randolph, and Brian Kuhlman. Transfer learning to leverage larger datasets for improved prediction of protein stability changes. *Proceedings of the National Academy of Sciences of the United States of America*, 121(6): e2314853121, 2024. doi: 10.1073/pnas.2314853121.
48. Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.

Supplementary Methods

Experimental target data and target-label scaling

All target-task comparisons use the same NbBench thermo-seq endpoint (25). The processed tables hold an amino-acid sequence field and a melting-temperature label in degrees Celsius. The split sizes are 57 training, 114 validation, and 396 held-out test sequences. As described in Methods, we deliberately assigned the smallest NbBench partition to training and the largest to the test set so that the model sits in the low-data regime the study targets.

Split definitions. The splits are fixed throughout. The 57 training examples fit network parameters, the validation split drives checkpoint selection, early stopping, and choice among candidate settings, and the test split stays untouched until those choices are fixed. The same rule applies to Tm-only models and to models trained with computational labels.

Target-label scaling. Before optimization, the training workflow maps the experimental Tm values to a scaled regression target. The scaler is fit only on the 57 training labels, using robust scaling followed by linear rescaling to [0,1]. Validation labels are transformed with this training-fitted scaler during model selection. Test predictions are averaged across seeds in scaled-label space and inverse-transformed to degrees Celsius before errors are computed, so the validation and test label distributions never influence the fitted transform.

Experimental-label scaling reference. This reference uses the same training split but subsamples it to the requested number of labels with the run seed; the validation and test splits are never subsampled. It serves only as a positive control for the pipeline. Adding true target labels should lower the Tm prediction error.

Processed computational-label tables

Computational labels are loaded as computational tasks rather than appended to the Tm data as extra measurements. Table 1 lists the processed target and computational-label files used in the reported comparisons. Mutation-effect tables carry sequence fields and [0,1]-scaled computational-label fields. The MD native-contact tables use the same processed format so that they can enter the same computational-task loader; there, the scaled label is a [0,1]-scaled native-contact fraction rather than a mutation free energy. The FEP-matched MD table (the primary MD label) is built on the same 1MEL/4IDL mutation scans as FEP; the diverse-panel MD table is used only as the negative data-design control.

Table 1. Processed target and computational-label data. Row counts exclude header rows.

Data type	Role	Processed rows	Use in reported comparisons
Experimental Tm	Target train	57	Model fitting
Experimental Tm	Target validation	114	Checkpoint and setting selection
Experimental Tm	Target test	396	Final held-out evaluation
FEP mutation free energy	Computational tasks	435 + 409	Main computational task
MD native-contact (FEP-matched scan)	Computational tasks	431 + 406	Matched-design MD label (primary)
Rosetta mutation score	Computational tasks	435 + 409	Computational-label comparison
ThermoMPNN stability score	Computational tasks	435 + 409	Computational-label comparison
Random variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
ESM2-proposed variants scored by Rosetta	Computational tasks	1000 + 1000	Computational-label comparison
MD native-contact (diverse panel)	Computational task	1143	Negative data-design control

Computational-task assignment. The loader assigns each row to a target or computational task. Experimental Tm rows use the Tm head. In the main mutation-effect model, the two template-specific tables use separate computational heads, and the MD-derived Q-value table uses a single computational head. This assignment fixes which regression head an example uses and which task-specific loss term it contributes to.

Sequence-overlap check. We checked exact sequence overlap between the processed computational tables and the fixed NbBench splits. The FEP, Rosetta, and ThermoMPNN mutation-effect tables, the FEP-matched MD native-contact table, and the random and ESM2-proposed variant tables — all built on the 1MEL and 4IDL scaffolds — had no exact match to any NbBench training, validation, or test sequence. The diverse-panel SAbDab MD table contained exact matches to 2 NbBench training, 4 validation, and 8 test sequences. We disclose this because it bears on the negative data-design control; because that diverse-panel table is used only as the negative control and does not enter any positive result, the overlap cannot account for any reported gain.

Computational-label sampling. For mutation-effect experiments with requested count n, the loader samples up to n rows independently from each of the two template tables using the run seed; the final FEP setting with $n = 320$ therefore supplies 320 sampled 1MEL rows and 320 sampled 4IDL rows before the computational-task split. The FEP-matched MD native-contact label samples from the same two template-specific tables. The diverse-panel MD control samples n rows from its single table. Computational-label rows are split 80/20 into computational-training and computational-validation rows with the same seed.

The production training call then filters the validation data passed to the trainer so that checkpoint selection and early stopping depend only on the experimental Tm validation examples.

Diverse-panel MD control: SAbDab structure selection

The following describes the *diverse-panel* MD native-contact control (the negative data-design control of Fig. 3). The FEP-matched MD label used for the positive results is built on the 1MEL/4IDL mutation scans as described in Methods. The diverse-panel MD control was built from SAbDab, the Structural Antibody Database (27). The archived SAbDab nanobody summary table holds 2422 rows with PDB identifier, heavy- and light-chain assignments, model index, antigen annotation, experimental method, resolution, species, and other fields. The manifest workflow matches this summary against the archived IMGT-renumbered PDB files.

Structure-selection rule. The rule is deterministic. For each PDB identifier with both a SAbDab summary row and an IMGT-renumbered PDB file, the workflow takes the first heavy-chain row in the SAbDab table and accepts the chain only if that chain identifier is present in the IMGT PDB file. The output manifest holds one entry per PDB identifier, with fields for the manifest row number, PDB identifier, selected heavy chain, SAbDab TSV row, IMGT PDB path, and MD run directory. This initial one-per-PDB manifest contained 1177 structures.

400 K simulation panel. The 400 K campaign reused the systems prepared in the 300 K campaign and built a 1146-entry 400 K manifest, consisting of 798 X-ray, 345 electron-microscopy, and 3 NMR structures according to the SAbDab method field. None of the selected rows had an annotated light chain, consistent with a single-domain antibody panel. After trajectory and Q-value processing, 1143 unique PDB identifiers yielded usable computational-label rows.

MD system preparation and simulation protocol

System preparation. Each structure was prepared as a nanobody-only system. The workflow takes the IMGT-renumbered PDB as the input structure, extracts the selected heavy chain, keeps only protein atoms, and leaves the termini uncapped, at pH 7.4. Chains are cleaned with PDBFixer, and pH-aware protonation is assigned through pdb2pqr/PROPKA when available (33, 34), with a fallback to AmberTools-based hydrogen assignment and Amber naming. The prepared chain is solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topology and coordinate files are built with ff19SB and OPC (29–31).

300 K equilibration branch. The base 300 K branch uses OpenMM on CUDA devices (32). It runs 1 ns of NVT followed by 1 ns of NPT at 300 K and 1 atm, with a 2-fs timestep and no hydrogen-mass repartitioning. Production at 300 K was generated for the broader campaign, but the computational label used here comes from the 400 K branch.

400 K production branch. The first 400 K relaxation stage restarts from the 300 K equilibrated state and runs 1 ns of NVT at 400 K with Cα restraints; the second repeats 1 ns of restrained 400 K NVT. Both relaxation stages use a 2-fs timestep and no hydrogen-mass repartitioning. Production runs 100 ns of restraint-free 400 K NVT, with hydrogen-mass repartitioning, 4 amu hydrogen mass, a 4-fs timestep, and coordinate/energy output every 100 ps.

Integrator and reporting settings. In OpenMM, explicit-solvent periodic systems use PME electrostatics with a 1.0-nm nonbonded cutoff and constraints on bonds to hydrogen. NPT stages use a Monte Carlo barostat; NVT stages omit it. The integrator is the Langevin middle integrator with 1 ps^{-1} friction. The 100-ps reporting interval gives 1000 expected frames for a complete 100 ns production trajectory.

MD-derived Q-value extraction

Trajectory input requirements. The processed MD table was generated from the archived 400 K production trajectories. A system was eligible only if it had a production trajectory, a prepared reference structure, and an Amber parameter file. The amino-acid sequence was read from the prepared structure using the first Cα atom of each residue, and sequences shorter than 50 residues were excluded.

Native-contact definition. Q-values are computed with MDTraj (35). The topology is loaded from the Amber parameter file, and the native reference is frame 0 of production. The calculation uses protein backbone heavy atoms. A candidate native contact must span more than three residues, have a reference-frame distance below 0.45 nm, and include at least one atom from a hydrophilic residue (Asp, Glu, Gln, Asn, Arg, Lys, or His, including Amber protonation variants). For a native-contact pair (i, j) with reference distance r^0_{ij} and instantaneous distance r_{ij}, the per-frame contribution is

$$q_{ij} = \frac{1}{1 + \exp\{\beta(r_{ij} - \lambda r^0_{ij})\}},$$

with $\beta = 50$ nm^{-1} and $\lambda = 1.8$, following the smooth native-contact form of Best-Hummer-style Q-value analyses (28). The frame-level Q-value is the mean of q_{ij} over all selected native contacts.

Trajectory-level Q-value label. The trajectory-level label is the mean Q-value over the terminal portion of production. For trajectories with more than 300 frames, the implementation uses the larger of 300 frames and one third of the available frames;

for shorter trajectories it uses all frames. For a complete 100 ns trajectory with 100-ps output, this is 333 terminal frames, roughly the final 33 ns. In the processed 400 K table, the number of frames used ranges from 66 to 333 (median 333), the number of native contacts from 28 to 728 (median 173), and the sequence length from 58 to 461 residues (median 122). Raw Q-values range from 0.0437 to 0.9994 and are scaled to [0,1] for computational-task training.

Sequence tokenization and encoder inputs

Sequences are tokenized with the ESM2 tokenizer for the selected encoder (14), using truncation and fixed-length padding to 160 residues. In the production protocol, the trainer receives all target and sampled computational-label rows for training, but its evaluation data are filtered to the experimental Tm validation rows.

Hot-encoder input path. The hot-encoder setting keeps tokenized sequences in the training data and updates the ESM2 weights. The frozen-encoder setting runs ESM2 once per target and computational-label sequence to precompute first-token embeddings, after which the optimizer updates only the regression trunk and task heads.

Neural-network architecture

The default encoder is the 8M-parameter ESM2 model, and the model uses its first-token representation as the sequence-level embedding. The shared trunk has dimensions hidden-size $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 32, with ReLU activations and dropout after the first two hidden layers. The Tm head maps the 32-dimensional representation to one scaled scalar prediction.

Target and computational heads. The Tm-only baseline is the encoder, the shared trunk, and the Tm head. Computational-label transfer models keep this target path and add computational heads. The main mutation-effect model uses separate computational heads for the two template-specific tasks. The code also implements computational-head controls, including a single shared mutation-effect head, a template-conditioned head, and a template-specific affine calibration on top of a shared head, to test whether the two template-specific computational labels need separate output calibration.

Architecture sensitivity analyses. Architecture controls include residual, latent, and mixture-style computational-fusion variants, in which the computational path produces a correction to the Tm prediction, a latent interaction between Tm and computational features, or a gate between a target-only expert and a computational-informed expert. The main reported FEP result uses the original shared-trunk architecture with separate template-specific computational heads; the other variants are kept as sensitivity analyses, not as the main claim.

Training objective, optimizer, and checkpoint selection

Each training example is a sequence, a scaled label, and a task identifier, and contributes loss only through the regression head for its task. The per-task loss is Huber loss with $\delta = 1.0$ on the scaled label. The default multi-task objective uses learned task-uncertainty weights (48). For task k with loss L_k and learned log-scale s_k, the contribution is

$$\frac{L_k}{2\exp(2s_k)} + s_k,$$

and the total loss sums the contributions from the tasks present in the minibatch. Fixed computational-loss weights are also implemented and were included in the candidate searches for computational classes where loss balance could shift validation performance.

Optimizer and schedule. The optimizer is AdamW. The default head/trunk learning rate is 3×10^{-4}, the default hot-encoder learning rate is 1×10^{-4}, weight decay is 0.04, dropout is 0.15, batch size is 16, warmup is 100 steps, and the schedule is cosine decay over at most 400 epochs. Checkpoints are evaluated and saved once per epoch, and mixed precision is used when CUDA is available. In the hot-encoder setting, the trainer creates separate optimizer parameter groups for ESM2 parameters and for the freshly initialized trunk/head parameters; in the frozen-encoder setting, ESM2 parameters are non-trainable and no encoder group is used.

Checkpoint selection. Within each run, the best checkpoint has the lowest mean squared error on the experimental Tm validation rows, and early stopping monitors the same metric with patience 15 and threshold 0. Computational-task validation rows are not passed to the trainer in the production protocol. This detail is central to the comparison. Computational labels shape the learned representation through the training loss, but they do not decide which checkpoint is used for target-task prediction.

Candidate-setting selection and final evaluation

Candidate settings are selected on the experimental Tm validation split, tuning every source independently and separately for the frozen and hot encoder regimes with a two-stage search. Stage 1 fixes learning rate, dropout, and weight decay at defaults and searches the model architecture (shared trunk, residual fusion, or latent fusion) crossed with the auxiliary-head coupling (separate template-specific heads or a template-conditioned shared head). Stage 2 holds the best Stage 1 skeleton and searches, for the hot regime, the encoder learning rate (1×10^{-4}, 5×10^{-5}, 3×10^{-5}, 1×10^{-5}) and, for the frozen regime, the head learning rate (1×10^{-3}, 5×10^{-4}, 2×10^{-4}), each crossed with dropout (0.05, 0.15, 0.30) and weight decay (0.02, 0.10). This staged search covered 30 Stage 1 and 126 Stage 2 configurations for the core sources, and an additional 48 Stage 1 and 168 Stage 2 configurations for the structure-based comparators.

Seeded candidate evaluation. Each candidate is evaluated as a three-seed ensemble on the Tm validation split at 320 auxiliary labels, and the single best configuration for each source and regime is then evaluated on the held-out test split as a five-seed ensemble. The run seed controls Python, NumPy, PyTorch, and CUDA randomness when CUDA is available, as well as target-training subsampling in the target-label scaling experiment, computational-row sampling, and computational-label train/validation splitting. The label-count curves use the selected per-source configuration while the number of computational labels is varied over 20, 80, 160, and 320.

Statistics and stored outputs

The primary metric is MAE in degrees Celsius on the 396 held-out test examples. For each condition, the five seeded models produce scaled Tm predictions for every test sequence; these are averaged in scaled-label space and inverse-transformed to degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations are also computed, but the figures emphasize MAE because it reads directly as an average temperature error.

Bootstrap intervals. Single-condition confidence intervals come from nonparametric bootstrap resampling over test examples. The absolute-error vector from the five-seed ensemble is resampled with replacement 10,000 times using a fixed bootstrap seed, and the reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. The 90% interval is used for consistency with the directional paired comparisons in the main text; the stored bootstrap summaries allow other interval levels to be recomputed. Paired comparisons use the same bootstrap indices for the candidate and reference absolute-error vectors. The reported ΔMAE is candidate minus reference, so negative values mean lower error than the reference.

Stored run summaries. Each training and evaluation call writes a structured summary recording the command-line arguments, relevant environment variables, resolved model settings, per-label-count metrics, bootstrap intervals, per-example absolute errors, and paired bootstrap summaries. Compact tables derived from these summaries feed the main computational-label comparison, the frozen-encoder controls, the computational-head controls, and the computational-combination controls.

Interpreting the label-count curves

Target-label scaling reference. The target-label scaling curve is a positive control on the pipeline. It tests whether the same model family and evaluation code recover the expected gain when more true experimental Tm labels are available. The FEP computational-label curve tests something different. It asks whether more labels from a physically related but distinct computed quantity improve the same held-out Tm endpoint, while the FEP-matched MD native-contact curve tests whether a structural stability observable transfers once its data design matches the target. The decisive comparisons are therefore not "simulation versus no simulation," but whether the auxiliary label shares the target's sequence design and whether its physical observable transfers under target-centered model selection.

Supplementary figure data tables

The supplementary figures are generated from tabular computational-label files derived from the same run summaries as the main figures. The processed computational tables, rendered supplementary figures, and figure-generation code will be distributed with the public analysis archive.

Fig.-to-data and code mapping

The manuscript figures are regenerated from compact result summaries and computational-label tables rather than from scratch run directories. The mapping below is the first place to check when auditing or extending a figure.

Table 2. Manuscript Fig.-to-data mapping. The table identifies the processed summaries and scripts used to regenerate each display item.

Display item	Inputs	Script	Primary comparison
Fig. 1	Conceptual protocol	`plot/make_outline_figures.py`	Target-centered transfer-learning design
Fig. 2	Diverse-panel vs matched-scan MD summaries; frozen label-count sweeps	`plot/make_outline_figures.py`	Data-design axis: same MD observable null as a diverse screen, positive as a matched scan
Fig. 3	Tuned frozen/hot per-source summaries	`plot/make_outline_figures.py`	Physical-observable axis: FEP transfers in both regimes, MD native-contact only frozen
Supplementary Figs. 1 to 5	Supplementary manifest and generated TSV tables	`plot/make_supplementary_figures.py`	Data-set composition, candidate-setting searches, model controls, count-sweep controls, and MD-feature controls

Generated supplementary tables. The final computational-label screen settings used in Figs. 3 and 4 and the frozen-encoder controls are recorded in generated supplementary tables, which list the selected architecture, validation-selected setting, computational-label count, validation MAE, held-out test MAE, bootstrap interval, and run-summary reference for each condition. A panel-level mapping table for all supplementary figures is included in the public analysis archive.

Supplementary Figs.

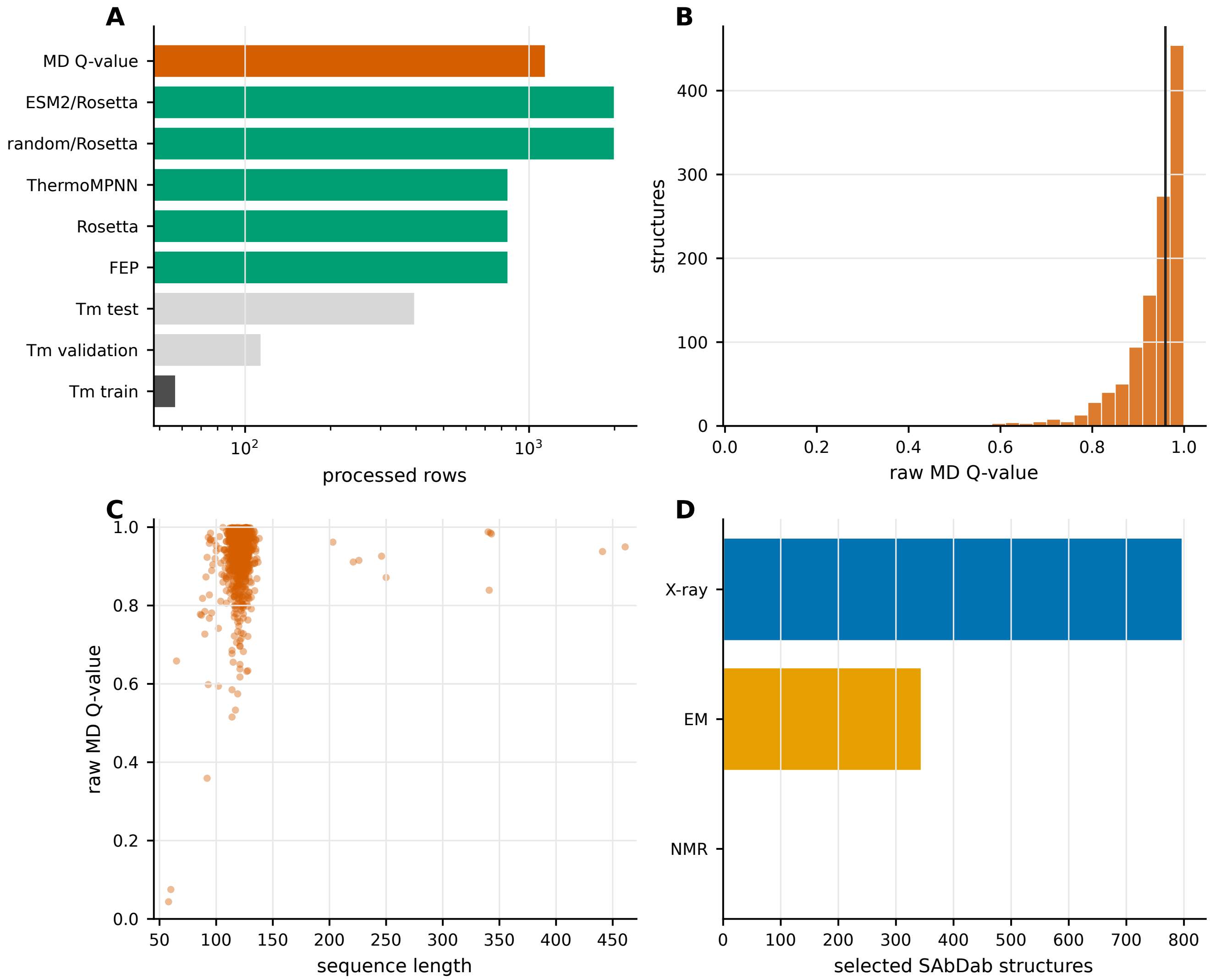

Fig. 4. Supplementary Fig. 1. Processed data sets and MD-derived Q-value labels. (a) Processed target and computational-label row counts used in the reported experiments. **(b)** Distribution of raw MD-derived Q-values before [0,1] scaling. **(c)** Relationship between sequence length and raw MD-derived Q-value. **(d)** Experimental methods for the selected SAbDab structures used in the 400 K MD panel.

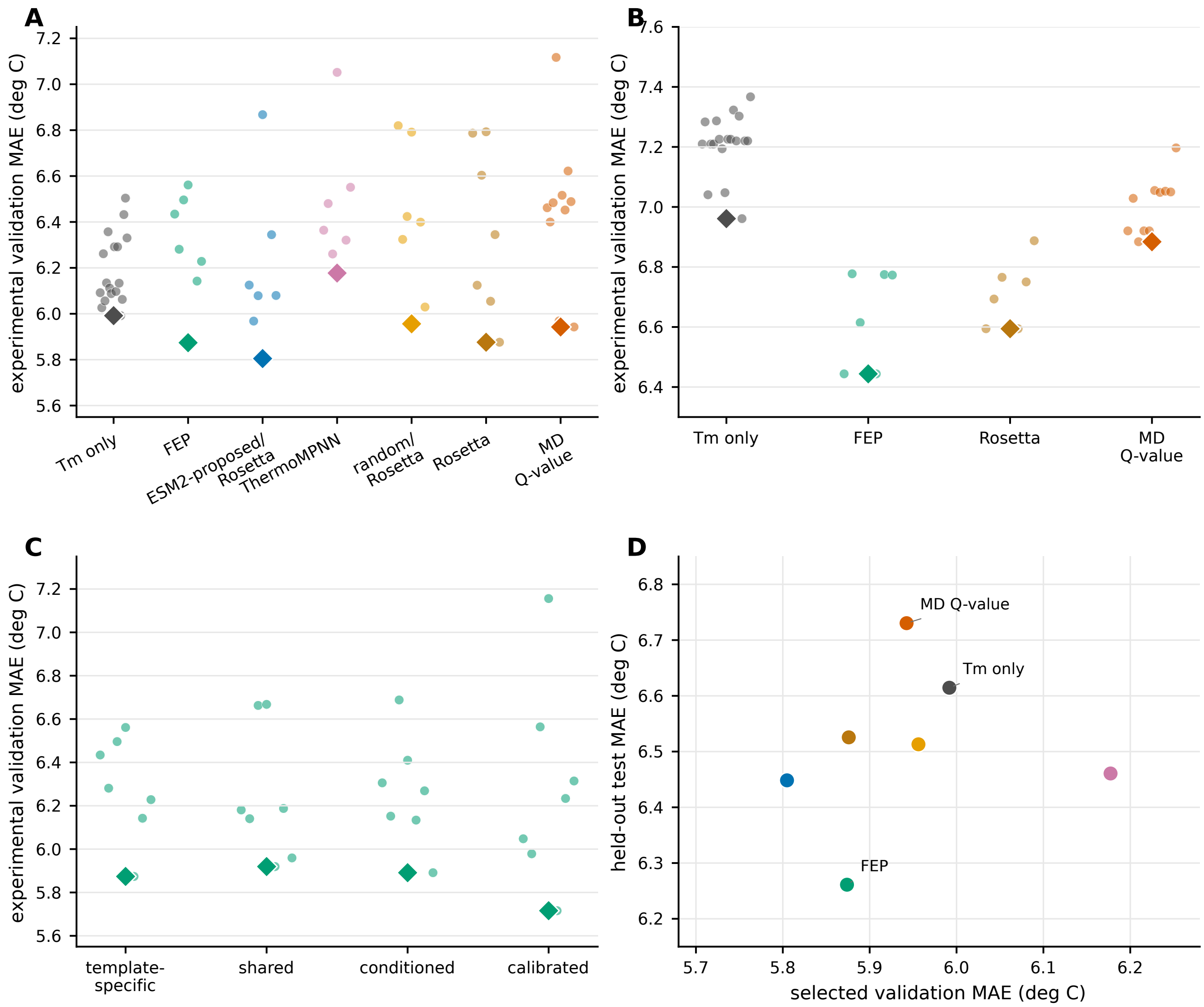

Fig. 5. Supplementary Fig. 2. Candidate-setting searches and target-validation selection. (a) Experimental validation MAE values for hot-encoder computational-label comparisons. **(b)** Experimental validation MAE values for frozen-encoder controls. **(c)** Candidate computational-head designs for FEP mutation free-energy labels. **(d)** Relationship between selected validation MAE and held-out test MAE for the final computational-label comparison. Diamonds mark the best validation setting in panels a to c.

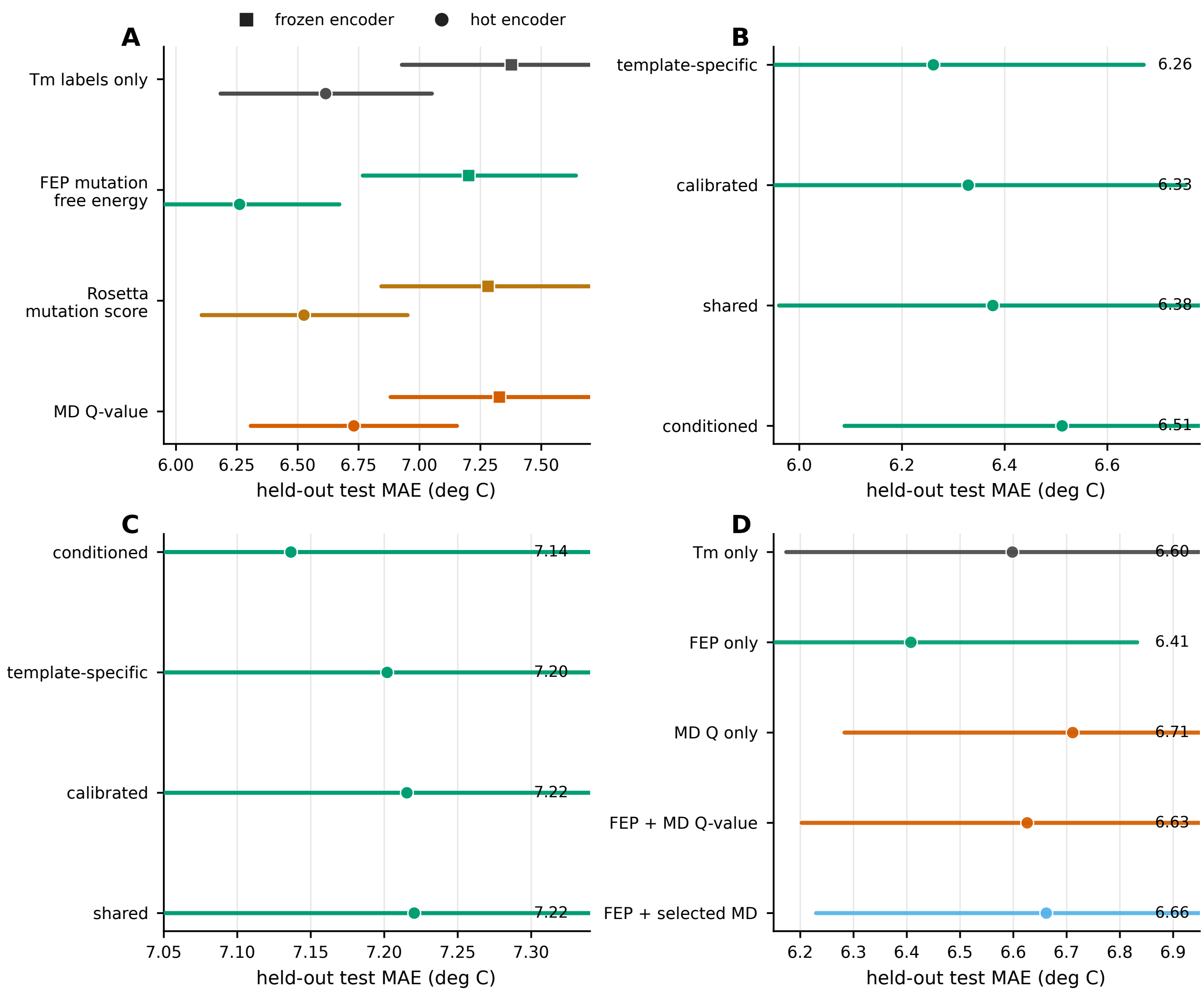

Fig. 6. Supplementary Fig. 3. Model and computational-combination controls. (a) Frozen-encoder and hot-encoder comparisons for selected computational labels. **(b)** Hot-encoder FEP computational-head controls. **(c)** Frozen-encoder FEP computational-head controls. **(d)** Computational-combination controls comparing Tm-only, FEP-only, MD Q-value-only, and combined-computational-label settings.

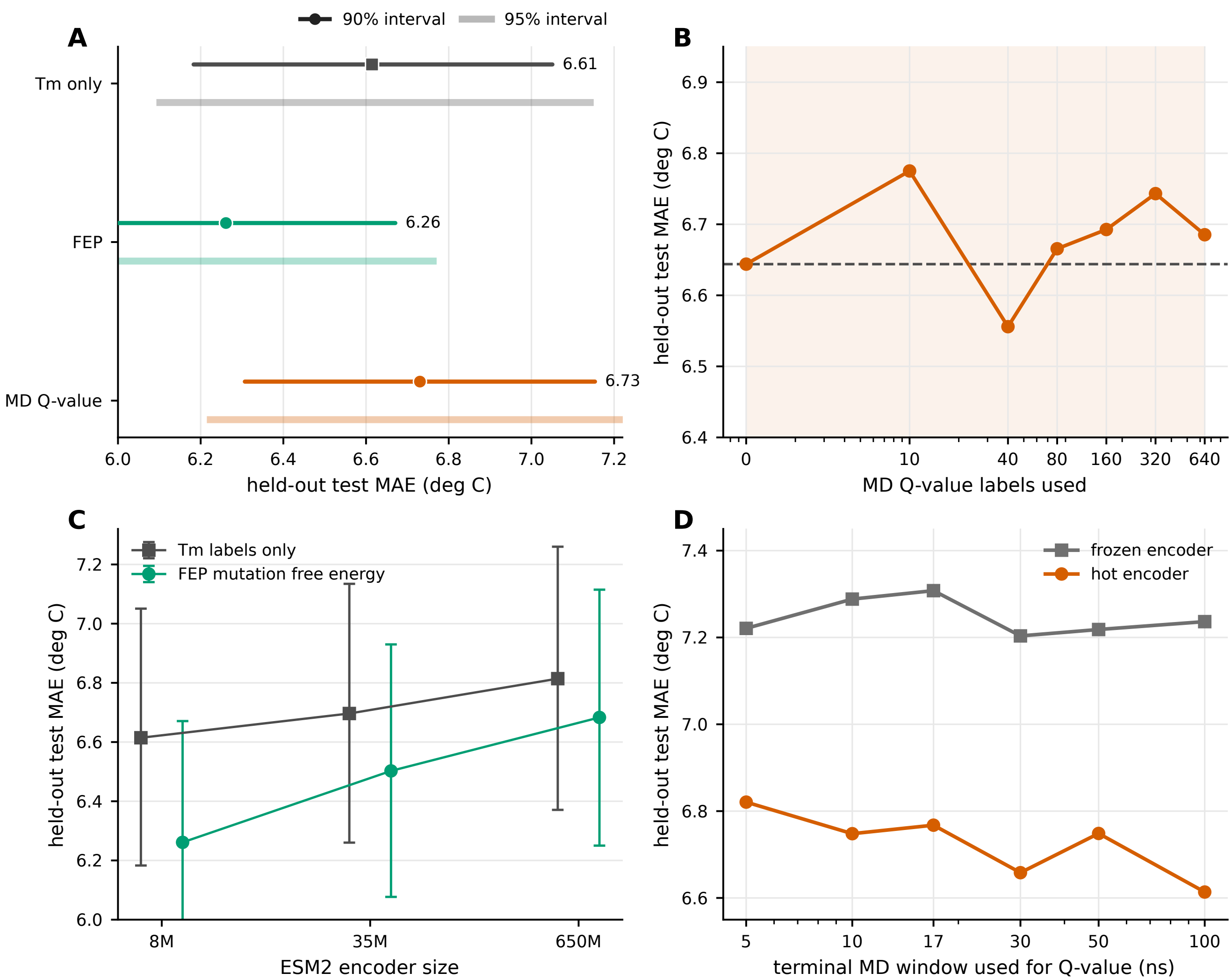

Fig. 7. Supplementary Fig. 4. Interval, scaling, encoder-size, and MD-window controls. **(a)** Final Tm-only, FEP, and MD Q-value conditions shown with 90% and 95% bootstrap intervals. **(b)** MD Q-value label-count curve after selecting the candidate setting separately for each label count on the experimental validation split. **(c)** ESM2 encoder-size controls for Tm-only and FEP settings. **(d)** MD trajectory-window controls for Q-value labels.

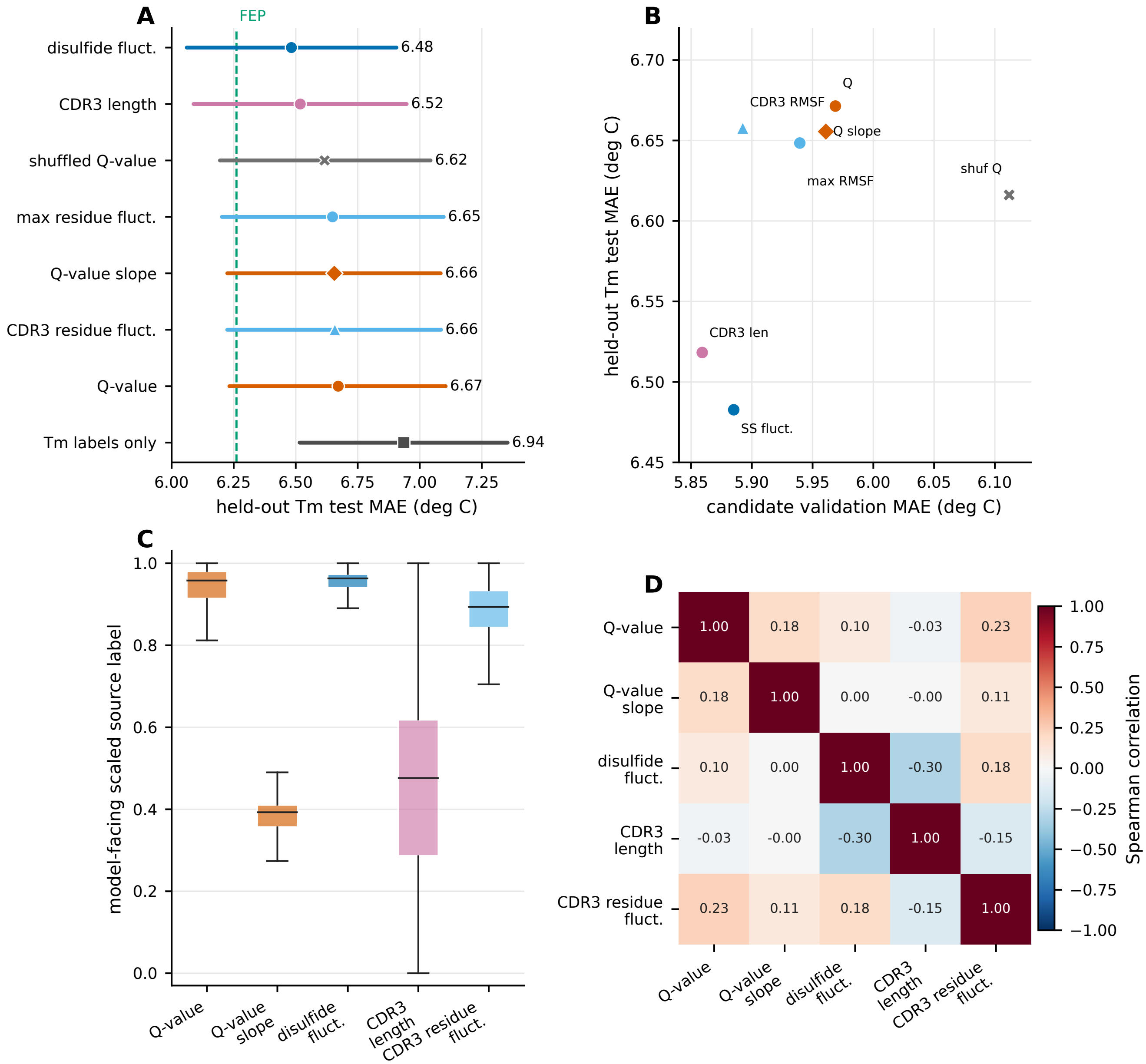

Fig. 8. Supplementary Fig. 5. Descriptor controls for MD-derived computational labels. (a) Held-out test MAE values for selected MD-derived and nanobody-specific descriptor controls. The dashed line marks the FEP reference. **(b)** Relationship between candidate validation MAE and held-out test MAE for descriptor controls that were carried forward to final evaluation. **(c)** Distributions of the model-facing scaled computational labels for the selected descriptors. **(d)** Spearman correlations among the selected computational labels over shared sequences.